QoS-Enabled Wireless Split Federated Learning: A Reinforcement Learning and Optimization Approach

Latif U. Khan, *Member, IEEE*, Maher Guizani, *Member, IEEE*, Sami Muhaidat, *Senior Member, IEEE*, and Moussa Ayyash, *Senior Member, IEEE*

***Abstract*—Federated learning (FL) has various advantages, including reduced communication overhead and improved privacy protection for situations involving frequent data production. FL involves training local models on end devices before transferring them to the cloud or network edge for global aggregation. End-devices can use this aggregated model to enhance their local models. Until convergence is reached, this iterative procedure is carried out repeatedly. FL has many benefits, but it also has several drawbacks. Constraints in computing resources are the most notable. In general, end-devices lack the computational capacity to learn local models effectively. To solve this issue, the split FL (SFL) was developed. However, wireless resource limitations and uncertainty also make SFL difficult to work. For SFL at the network edge, we consider. a joint task-offloading, resource allocation, and end-devices computing resources optimization problem. Moreover, we consider the quality of service (QoS) constraint in terms of latency for SFL. Because our problem involves both continuous and binary variables, it is a mixed-integer non-linear programming problem and is challenging to solve. We provide a solution based on optimization and a double deep Q-network (DDQN) with dueling. We use comprehensive simulations to validate the proposed approach. Our proposed scheme, namely, DDQN+dueling, outperforms traditional DDQN-based schemes in terms of faster convergence and attaining QoS for various configurations.**

***Index Terms*—Federated learning, split federated learning, double deep Q-network, resource-constrained consumer electronic devices.**

I. Introduction

IN ORDER to achieve superior privacy-aware machine learning for a variety of applications, federated learning (FL) has recently been the subject of substantial research [1], [2], [3]. In FL, devices run local models iteratively and migrate their resultant local model to the nearby edge or remote cloud, depending on the configuration, to participate in the aggregation process. At the edge or cloud, aggregation of local models from various nodes happens. Next to aggregation, the global model obtained in the aggregation process is shared with end-devices to further update their local models. This process takes place iteratively until a convergence is attained. FL is significant in wireless systems primarily for two reasons. These include improved privacy protection and effective communication learning for situations where data generation occurs often (autonomous cars, for example, create 40, 000 gigaoctet of data every day). As a result, sending regularly generated data to the cloud under these situations is quite difficult. FL works well in these kinds of situations. FL has limitations, even though it makes numerous advantages possible. Due to limited computational resources, training local FL models at end devices is difficult. Furthermore, it is difficult and entails numerous limitations to share learning updates over a wireless channel. The FL learning process will become less effective as a result of these deficiencies.

In order to overcome the difficulty of computing resource constraints, split FL (SFL) was introduced in [4]. In [4], split FL (SFL) was introduced as a solution to the computing resources constraint problem in FL. The performance of SFL and FL over wireless networks was evaluated in the work in [5]. An additional study in [6] suggested a hybrid SFL and FL architecture. For their framework, they modeled wireless communication, and they used the simulation findings to do important analysis. A particular federated split learning architecture was designed and results were given in the study in [7]. The use of federated learning, split learning, and transfer learning for intelligent transportation system applications was presented by Otoum et al. in [8]. We propose a novel dueling-based double deep Q-network for effective task-offloading and resource allocation in SFL over wireless networks, which differs from the works in [5], [6], [7], [8]. The summary of our contributions are as follows:

- *Joint task offloading and resource allocation framework:* We present a joint task offloading and resource allocation framework for SFL over wireless networks with computing resource-constrained devices. This framework will enable us to partially compute the SFL model at the end-devices and then, offload the rest of the learning task to the network edge.

Received 31 December 2024; revised 27 April 2025; accepted 13 May 2025. Date of publication 8 July 2025; date of current version 7 November 2025. *(Corresponding author: Latif U. Khan.)*

Latif U. Khan is with the Department of Computer Science and Information Technology, Abu Dhabi University, Abu Dhabi, UAE (e-mail: latif.u.khan2@gmail.com; latif.khan@adu.ac.ae)).

Maher Guizani is with the Computer Science and Engineering Department, University of Texas at Arlington, Arlington, TX 76019 USA.

Sami Muhaidat is with the 6G Research Center, Khalifa University, Abu Dhabi, UAE.

Moussa Ayyash is with the Department of CIMST, Chicago State University, Chicago, IL 60628 USA.

Digital Object Identifier 10.1109/TCE.2025.3587176

- *QoS-aware problem formulation:* We design a cost function that counts for both latency and energy. Moreover, we formulate a problem whose objective is to minimize the cost function via optimizing the local computing resource variable, task offloading and resource allocation, and edge computing resource allocation. We introduced a QoS constraint in our problem formulation. This constraint is to ensure that latency in sharing of learning updates between the end-devices and edge servers should not exceed the maximum threshold.
- *Dueling-enabled DDQN for cost optimization of SFL:* Our problem has an NP-hard nature for many end-devices and edge servers. Therefore, it is very challenging for us to use directly conventional convex optimization techniques. Due to these limitations, we propose a solution based on double deep Q-network and convex optimization. For optimizing end-devices and edge servers computing resource, we use convex optimizers. On the other hand, for task offloading and resource allocation, we use DDQN. Although DDQN performs well for many scenarios, it might not be very stable and have good performance for highly dynamic scenarios. Therefore, we added the concept of dueling in DDQN to further improve its performance. Our dueling-based DDQN scheme offers significant performance improvement.
- *Results:* Finally, for a proof of concept, we present extensive results to support our idea of QoS-aware SFL.

II. Related Works

Many works [1], [4], [5], [6], [7], [8], [9], [10], [11], [12] considered SFL. An overview of recent works is given in Table I. The work in [5] considered both split learning and federated learning. Based on their analysis, they proposed SFL and analyzed it in detail. They validated their results for MNIST, CIFAR-10, FMNIST, and HAM10000 datasets. Another work [6] proposed a scheme, namely, hybrid SL and FL that uses both SL and distributed learning over wireless networks. They formulated an optimization problem and provided extensive convergence analysis. Finally, simulation results were provided for various settings. The work in [7] extensively studied both SFL and FL. Moreover, they provided extensive simulation results. The work in [8] proposed the use of SFL for intrusion detection system for intelligent transportation system. They used Canadian Institute of Cybersecurity Intrusion Detection System (CICIDS2017) dataset for their evaluation. According their analysis, their proposed scheme achieved a good accuracy and suitable for use in end-devices with resource constraints in IoT systems. Another work [9] proposed the framework, namely, RingSFL. RingSFL is based on split mechanism for better addressing client heterogeneity. In contrast to vanilla FL, where a local model is learned by an end-device, in RingSFL, a ring topology is formed. In a ring topology, the end-devices communicate with the server as well as with each other. Finally, the authors presented extensive simulation results for validating their proposal. The work in [1] proposed a hierarchical SFL that improves the convergence speed but at the cost of communication resources. In detail, the authors presented a framework along with a sequence diagram for flow of the process. They defined a cost function that accounts for both latency and energy. They optimized many parameters to minimize the formulated cost. These parameters are task offloading, resource allocation, end-devices computing resource management, transmit power allocation, and relative local accuracy. They used a block successive upper-bound minimization function for solution due to its power nature of solving non-convex problems. Moreover, they provided extensive convergence analysis. Another work in [10] used multi-agent DRL for model partitioning and aggregation control. The work in [11] increased the efficiency of SFL. The work in [12] considered resource scheduling and SFL. Different, as shown in Table I), from the works in [1], [4], [5], [6], [7], [8], [9], [10], [11], [12], our work focuses on the wireless communication aspect (i.e., specially we jointly consider task-offloading, resource allocation, and computing resource optimization) of SFL. In SFL, there will be many rounds of communications between devices and the edge servers. We will formulate a problem to minimize the training cost while satisfying the QoS in terms of latency. The latency should not be more than the maximum allowed threshold.

On the other hand, there are many works [13], [14], [15], [16], [17] on the use deep reinforcement learning for resource allocation and association in wireless networks. Xiong et al. in [13] proposed the use of deep reinforcement learning for resource allocation in IoT edge computing system. Their main objective is to minimize the long-term weighted sum of completion time of jobs. Specifically, they use deep Q-network with multiple replay memories. Another work [14] used deep reinforcement learning for resource allocation in blockchain-based resource allocation. The work in [15] considered multi-agent deep reinforcement learning for wireless networks resource management. They formulated a problem to maximize the objective function by optimizing users scheduling and power control decisions. Finally, they provided extensive numerical results. The work in [16] considered deep reinforcement learning for resource allocation in mobile edge networks. A cost function accounting for the latency was defined. Their formulated problem aimed at minimizing the cost function subject to many constraints. They proposed a scheme using deep deterministic policy gradient (DDPG) for resource management to minimize the long term reward. Similar to the previous works, the work in [17] proposed a Q-value iteration based reinforcement learning for resource management. Based on the works [13], [14], [15], [16], [17], it evident that we can use reinforcement learning for resource allocation and task offloading.

III. System Model and Problem Formulation

Fig. 1 illustrates a system of consumer electronics devices with limited resources. In our system, there are devices with limited computational power and thus they seem difficult to train local FL model. These devices are represented by the set $\mathcal{S}$ of S devices. There is a local dataset associated with each device s. These local datasets are denoted by a

TABLE I
COMPARISON OF RELATED WORKS WITH PROPOSED WORK

Reference	SFL	Task-Offloading	Multi-agent DRL	Dueling	Remark
Gao *et al.* [5]	✓	✗	✗	✗	Evaluated SFL for various distribution of data.
Liu *et al.* [6]	✓	✗	✗	✗	Proposed a hybrid architecture of SFL and considered client selection.
Turina *et al.* [7]	✓	✗	✗	✗	Analyzed the performance of SFL and FL for various settings.
Otoum *et al.* [8]	✓	✗	✗	✗	Considered SFL and transfer learning for securing ITS applications.
Thapa *et al.* [4]	✓	✗	✗	✗	Proposed SFL and tested it for various settings.
Shen *et al.* [9]	✓	✗	✗	✗	Combined SL with distributed learning.
Khan *et al.* [1]	✓	✓	✗	✗	Combined SFL and hierarchical learning for learning performance improvement.
Fan *et al.* [10]	✓	✗	✓	✗	Proposed efficient SFL for resource-constrained devices.
Zhu *et al.* [11]	✓	✗	✗	✗	Optimized user-side workload and server-side computing resource allocation for SFL.
Ahmed *et al.* [12]	✓	✗	✗	✗	Considered SFL for resource recommendation in smart cities.
Our paper	✓	✓	✓	✓	N.A

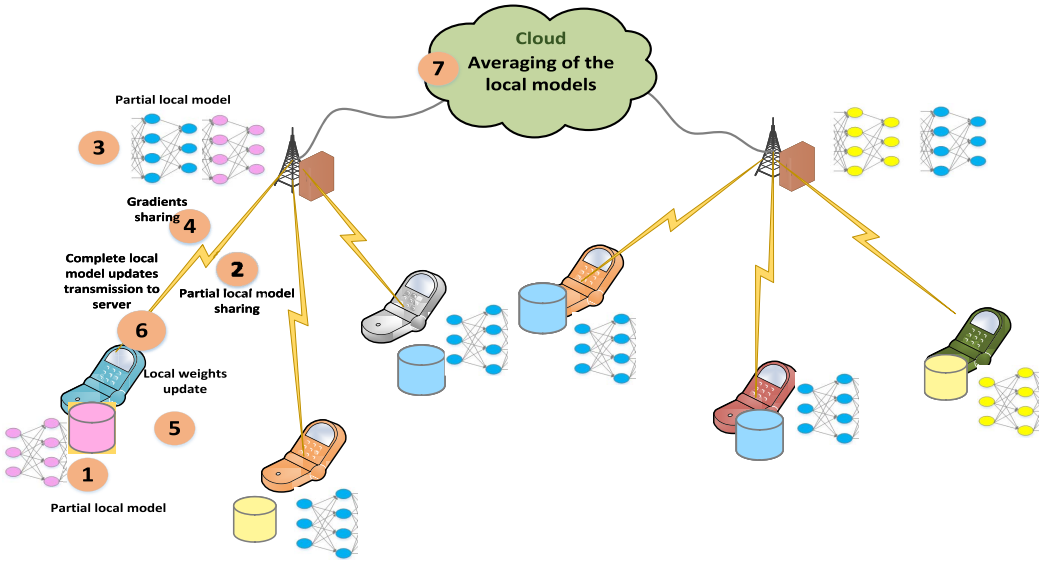


Fig. 1. Split federated learning system model.

set $\mathcal{B}_s, \forall s \in \mathcal{S}$. Every device s has B_s data points. These devices are aimed at participating in the learning process. However, they have limited computing resources. Therefore, these devices will compute a portion of the local model and request the edge servers to learn their remaining models. There is a set $\mathcal{L}$ of L edge servers for serving end-users. Note that both end-devices and edge servers have limitations in terms of computing resources. End-devices cannot completely learn local models, therefore, they learn a part of their models and share the partially learning model with the edge server. The edge server will further learn the partial model. This is the concept that is used by SFL. Even though SFL makes a lot of advantages feasible, end devices and edge servers must allocate wireless and computational resources effectively. Furthermore, the devices involved in SFL must effectively transfer their tasks to the edge servers. Here, first, wireless SFL model is discussed.

A. Wireless SFL Model

In our system, a set $\mathcal{S}'$ of S' resource-constrained consumer electronics devices want to take part in a global

TABLE II
SUMMARY OF THE KEY NOTATIONS

Notation	Description
$\mathcal{S}$	Set of end-devices
$\mathcal{B}_s$	Local dataset of device s
S	Total end-devices
$\mathcal{L}$	Set of edge servers
L	Total edge servers
Ξ_s^{local}	Local energy consumption at device s
ℓ_s^{local}	Local latency of device s
$\wp_s$	Computing resource assigned to device s
$\wp_{max}$	Upper limit of computing power
$\wp_{min}$	lower limit of computing power
$\wp_{MAX}$	Total computing power available for end-devices in a system
$\boldsymbol{\kappa}$	Task offloading matrix
$\boldsymbol{\upsilon}$	Resource allocation matrix
θ	Relative local accuracy
ℓ_{trans}	Transmission latency
ℓ_{trans}	Transmission latency
$\mathcal{A}_s$	Set of actions
$\mathcal{P}$	State transition probability.
Δ_s	Reward of device s.

model's learning process. Due to limitations in communication resources, only the set $\mathcal{S}$ of S will be included in the learning process among $\mathcal{S}'$ of S'. Because of limitations in computing power, each device learns a partial local model. Let $\mathcal{T}_s(c_s, b_s, t_s)\forall s \in \mathcal{S}$ be the learning task, where c_s and b_s represent the computational resources required for learning a specific local model for a single data point and entire local dataset points, respectively. The local latency in computing a partial model for a specific local model architecture is determined by:

$$\ell_s^{\text{local}}(\boldsymbol{\wp}) = \frac{c_s b_s}{\wp_s}, \forall s \in \mathcal{S}, \tag{1}$$

where the computational resource allocated to device s is indicated by $\wp_s$. Increasing local computing resources can lower latency, but at the expense of high local energy consumption (i.e., energy is proportional to the square of the device's running frequency)

$$\Xi_s^{\text{local}}(\boldsymbol{\wp}) = \alpha_s c_s b_s \wp_s^2, \forall s \in \mathcal{S}, \tag{2}$$

where α_s represents device s's effective capacitance coefficient. However, the processing resources of the local device should adhere to the limitations.

$$\wp_{min} \leq \wp_s \leq \wp_{max}, \forall s \in \mathcal{S}, \tag{3}$$

where the minimum and maximum limits are indicated by $\wp_{min}$ and $\wp_{max}$, respectively. However, the total computing power of all devices should not be greater than $\wp_{MAX}$, the upper limit.

$$\sum_{s=1}^{S} \wp_s \leq \wp_{MAX}. \tag{4}$$

In addition to computing the partial local model, it is shared with the edge server, which performs the remaining local model computation. There are two methods for determining the partly remaining local model at the network edge. One way is to provide labels at the network edge and the other way is to share back the data to end-devices for computing the loss function using labels. In this work, we assume the output labels are available at the network edge. After the edge servers receive a partial local model, further computation takes place at the network edge. It will share the gradients with the devices in addition to computing at the edge. We take into consideration an orthogonal frequency division multiple access (OFDMA) for communication between end-devices and edge servers. Since the availability of communication resources is limited, it makes sense for SFL devices to reuse the resources that are currently in use by cellular users. We can calculate the transmission latency as:

$$\ell_{\text{trans}}^{s}(\boldsymbol{\kappa}, \boldsymbol{\upsilon}) = \left(\frac{\kappa_{(s,l)} \upsilon_{(s,r)} O_s}{T_r \left(\log_2 \left(1 + \frac{p_s h_{s,l}}{\sum_{c \in C} p_c h_{c,b} + N_o^2} \right) \right)} \right), \forall s \in \mathcal{S}, \tag{5}$$

where T_r and O_s are the bandwidth of resource block and learning data overhead, respectively. As we are reusing the resource blocks, therefore, there will be an interference which can be given by $\sum_{c \in C} p_c h_{c,b}$. Meanwhile, the noise power is given by N_o^2. $\upsilon_{(s,r)}$ and $\kappa_{(s,l)}$ are resource allocation and task offloading variables, respectively. The value of task offloading variable $\kappa_{(s,l)} = 1$, when the task is offloaded to the edge server and $\kappa_{(s,l)} = 0$, vice versa. Similar to the task offloading variable, the resource allocation variable $\upsilon_{(s,r)} = 1$ if the resource block r is allocated to device s and $\upsilon_{(s,r)} = 0$, vice versa. In our system, every edge server has a certain capacity to serve end-users/end-devices. Such a limitation can be given by:

$$\sum_{s=1}^{S} \kappa_{s,l} \leq \kappa_{\text{s}}^{max}, \forall l \in \mathcal{L}, \tag{6}$$

where the maximum limit of serving by edge server can be given by κ_{s}^{max}. Furthermore, the task of a single device should be offloaded to a single edge server.

$$\sum_{l=1}^{L} \kappa_{s,l} \leq 1, \forall s \in \mathcal{S}, \tag{7}$$

On the other hand, there are limited communication resources. Therefore, a resource block should not be assigned to more than one device.

$$\sum_{r=1}^{R} \upsilon_{s,r} \leq 1, \forall s \in \mathcal{S}. \tag{8}$$

For sending learning updates to the edge servers, wireless resource blocks are necessary. To send learning updates, the

number of resource blocks needed depends on the size of the learning data, which is determined by the complexity of the end-device model architecture. For an end-device, we employ a single resource block [18].

$$\sum_{s=1}^{S} \upsilon_{s,r} \leq 1, \forall r \in \mathcal{R}. \tag{9}$$

Now, write the transmission energy as:

$$\Xi^{s}_{\text{trans}}(\boldsymbol{\kappa}, \boldsymbol{\upsilon}) = \left(\frac{\kappa_{(s,l)} \upsilon_{(s,r)} O_s}{T_r\left(\log_2\left(1 + \frac{p_s h_{s,l}}{\sum_{c \in C} p_c h_{c,b} + N_o^2}\right)\right)} \right) p_s, \quad \forall s \in \mathcal{S}, \tag{10}$$

After computation, a small cell base station shares the local model with the remote cloud. The following equation can be used to determine the number of rounds between the end devices and the edge servers:

$$I_g(\varepsilon, \boldsymbol{\theta}) = \frac{\Gamma \log(1/\varepsilon)}{1 - \theta}, \tag{11}$$

where the local dataset dependent constant is Γ. The relative local accuracy is represented by θ. The interpretation of θ differs from that of the local accuracy. In contrast to greater values, lower values of θ are preferred, and vice versa. The global accuracy is ε. As can be seen from (11), the global rounds are dependent on θ; that is, more global rounds will often occur when θ is higher, and vice versa. Furthermore, the maximum delay should not be exceeded by the maximum threshold.

$$\left(\frac{\kappa_{(s,l)} \upsilon_{(s,r)} O_s}{T_r\left(\log_2\left(1 + \frac{p_s h_{s,l}}{\sum_{c \in C} p_c h_{c,b} + N_o^2}\right)\right)} \right) \leq \phi_{max}, \quad \forall s \in \int, r \in \mathcal{R}. \tag{12}$$

Equation (12) is the QoS constraint that ensures that delay should not be more than the maximum threshold. Therefore, there is a need for efficient task-offloading and resource allocation. Next, we present our problem formulation.

B. Problem Formulation

The first step in formulating a problem is to determine an objective function. The cost of training the SFL model serves as our goal function. In our situation, we write the objective function taking into account both energy and delay. Initially, we write energy consumption as follows: [1]:

$$\Xi_{\text{total}}(\boldsymbol{\kappa}, \boldsymbol{\upsilon}, \boldsymbol{\wp}, \boldsymbol{\theta}) = \sum_{s=1}^{S} (1 + \theta) \Xi^{s}_{\text{trans}}(\boldsymbol{\kappa}, \boldsymbol{\upsilon}) + \sum_{s=1}^{S} \Xi^{\text{local}}_{s}(\boldsymbol{\wp}), \tag{13}$$

The concept of θ is used in (13), to represent how relative local accuracy affects energy cost. Larger θ values will result in larger costs, and vice versa. We now write the overall delay as follows:

$$\ell_{\text{total}}(\boldsymbol{\kappa}, \boldsymbol{\upsilon}, \boldsymbol{\wp}, \boldsymbol{\theta}) = \sum_{s=1}^{S} \ell^{s}_{\text{trans}}(\boldsymbol{\kappa}, \boldsymbol{\upsilon}) + \sum_{s=1}^{S} \ell^{\text{local}}_{s}(\boldsymbol{\wp}) \tag{14}$$

Similarly for (13), one can rewrite (14) as:

$$\ell_{\text{total}}(\boldsymbol{\kappa}, \boldsymbol{\upsilon}, \boldsymbol{\wp}, \boldsymbol{\theta}) = \sum_{s=1}^{S} (1 + \theta) \ell^{s}_{\text{trans}}(\boldsymbol{\kappa}, \boldsymbol{\upsilon}) + \sum_{s=1}^{S} \ell^{\text{local}}_{s}(\boldsymbol{\wp}) \tag{15}$$

It is evident from (15) that the cost in terms of latency will increase with an increase in relative local accuracy. There are many ways to improve the relative local accuracy. One is to increase the number of local iterations. Additionally, one can use more complex local model. Generally, both of these approaches will enhance the performance but not necessary. Also, for both of the aforementioned schemes, they need additional computing resources. Now, we write the overall cost as:

$$C(\boldsymbol{\kappa}, \boldsymbol{\upsilon}, \boldsymbol{\wp}, \boldsymbol{\theta}) = \beta \ell_{\text{total}} + (1 - \beta) \Xi_{\text{total}}, \tag{16}$$

where β is the scaling constant, which can be used to scale the energy and delay proportion when computing the cost of learning the SFL model. The objective of our problem formulation is to reduce the total cost of learning the SFL model.

$$\begin{aligned} \mathbf{P-M}: \quad & \underset{\boldsymbol{\kappa}, \boldsymbol{\upsilon}, \boldsymbol{\wp}, \boldsymbol{\theta}}{\text{minimize}} \quad C(\boldsymbol{\kappa}, \boldsymbol{\upsilon}, \boldsymbol{\wp}, \boldsymbol{\theta}) \\ & \text{subject to:} \\ & (3), (4), (6)-(9), (12). \end{aligned} \tag{17}$$

The problem as formulated is a MINLP problem. Constraint (3) denotes that the local computing resources should follow the lower and upper limits. Equation (4) ensures that the total computing resources should not be more than the maximum available computing for all end-devices. Equation (6) ensures the maximum possible number of end-devices served by edge server. Equation (7) ensures a single device should be associated with only one edge server. Equation (8) limits the allocation of a maximum of single resource block to a single device. Equation (12) is about the QoS in terms of latency. This limits the delay of all devices should not exceed the maximum threshold and thus ensure a sufficient QoS while learning SFL model. The problem as posed is hard to solve directly. Furthermore, standard optimization strategies are not applicable. As a result, as discussed in the following section, we shall employ a decomposition-based scheme.

IV. Proposed Optimization and Multi-Agent Reinforcement Learning Solution

The formulated problem **P-M** is very hard to solve using conventional convex optimization. The reason for this complexity lies in the presence of both continuous (i.e., local computing resource variable) and binary variables (i.e., resource allocation and task offloading variables). Therefore, we will use a decomposition scheme. This scheme will allow us to separately solve local computing resource optimization and joint resource allocation and task offloading. First, we solve the local computing problem.

A. Devices Computing Resource Allocation Problem

In this section, we discuss our local computing resource allocation problem. One can write the local computing optimization problem as follows:

$$\mathbf{P-DR}: \quad \underset{\wp}{\text{minimize}} \quad C(\boldsymbol{\wp}) \tag{18a}$$

subject to:

$$\wp_{min} \leq \wp_s \leq \wp_{max}, \forall s \in \mathcal{S}, \tag{18b}$$

$$\sum_{s=1}^{S} \wp_s \leq \wp_{MAX}. \tag{18c}$$

The goal of problem **P-DR** is to reduce the latency of end devices when calculating partial local models. In problem **P-DR**, we have one continuous variable $\wp$. The nature of the formulated problem is convex. To prove the convexity, we can check the objective function and the constraints. If we take the twice derivative of the objective function and then check it for its range of values for possible inputs. For all inputs, the range of the twice derivative of the objective function will be greater than or equal to 0. Based on this, we can say that the objective function is convex. Meanwhile, the constraints are linear inequality constraints. Therefore, one can say that the problem is a convex optimization problem and can be solved using a convex optimizer as shown in Fig. 3.

B. Joint Task-Offloading and Resource Allocation Problem

In this section, we present a joint task offloading and resource allocation problem as follows:

$$\mathbf{P-TR}: \underset{\kappa,\upsilon}{\text{minimize}} \quad C(\boldsymbol{\kappa}, \boldsymbol{\upsilon}) \tag{19a}$$

subject to:

$$\sum_{s=1}^{S} \kappa_{s,l} \leq \kappa_{\mathrm{s}}^{max}, \forall l \in \mathcal{L}, \tag{19b}$$

$$\sum_{l=1}^{L} \kappa_{s,l} \leq 1, \forall s \in \mathcal{S}, \tag{19c}$$

$$\sum_{r=1}^{R} \upsilon_{s,r} \leq 1, \forall s \in \mathcal{S}, \tag{19d}$$

$$\sum_{s=1}^{S} \upsilon_{s,r} \leq 1, \forall r \in \mathcal{R}, \tag{19e}$$

$$\Omega_{s,r} \leq \phi_{max}, \forall s \in \int, r \in \mathcal{R}, \tag{19f}$$

$$\kappa_{s,l} \in \{0,1\}, \upsilon \in \{0,1\}, \tag{19g}$$

where $\Omega_{s,r} = \left(\frac{\kappa_{(s,l)} \upsilon_{(s,r)} O_s}{T_r \left(\log_2 \left(1 + \frac{p_s h_{s,l}}{\sum_{c \in C} p_c h_{c,b} + N_o^2} \right) \right)} \right)$. Convex optimization strategies are unable to solve the combinatorial problem **P-TR**. A strategy based on relaxation and decomposition can be used to solve **P-TR** [19]. Approximation errors in the conversion of binary task-offloading and resource allocation variables into continuous and then back to binary variables are an inherent limitation of this approach. Using an exhaustive search technique is another method to get around this limitation [20]. The exhaustive search approach, however, has a significant computational complexity. Therefore, we will use alternate scheme, i.e., reinforcement learning based scheme.

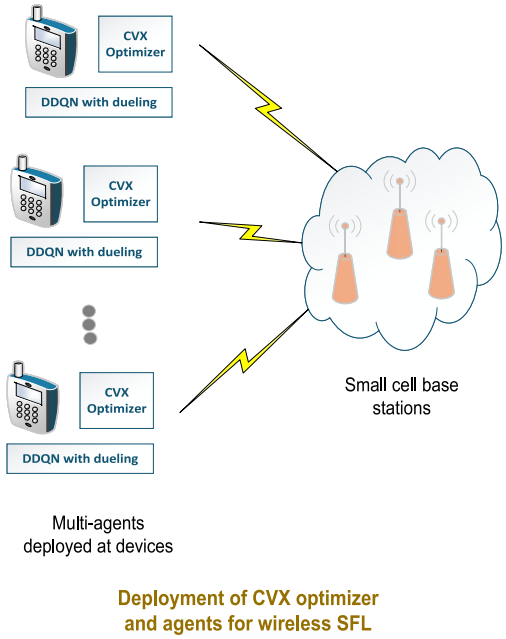


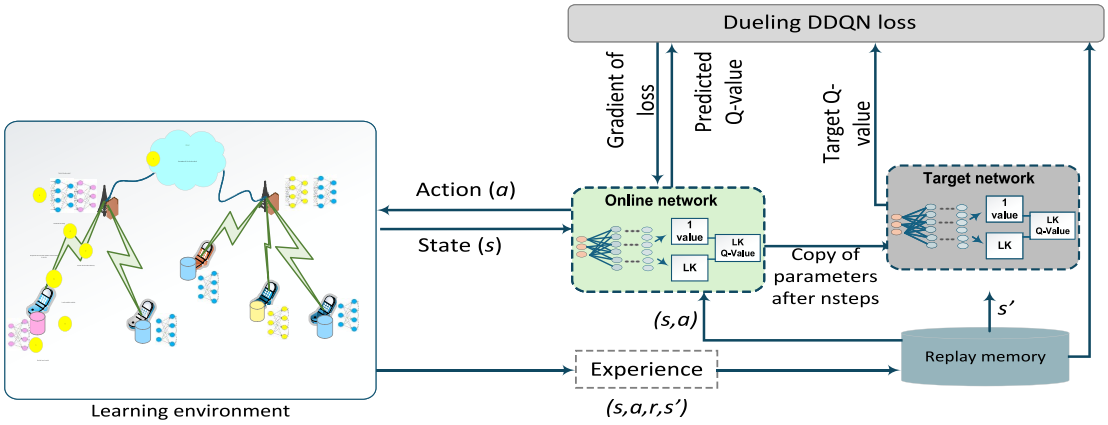
Fig. 2. Deployment of agents and CVX optimizers.

To solve problem **P-TR**, we will use multi-agent deep reinforcement learning (MARL), whose deployment is shown in Fig. 2. Various agents are deployed at the end-devices involved in SFL. The goal of MARL is to maximize the long-term reward. In problem **P-TR**, we want to minimize the cost function. On the other hand, MARL is based on maximizing the long-term reward. Therefore, we write the reward function as $\Delta_s = \frac{1}{C}$. In MARL, the long-term reward is the summation of the instantaneous rewards of various devices.

$$\Phi_s = \sum_{t=0}^{T-1} \gamma^t \Delta_s(t) \tag{20}$$

where γ^t denotes the discount rate that shows the weight of future reward. For $\gamma^t = 0$, it is evident that we are only considering a current reward without future rewards. For $\gamma^t \leq 1$, it evident that the future rewards less important compared to previous rewards in computing the long-term reward and vice versa. For using MARL, there is a need for formulation as a stochastic game. In our system, all the users want to maximize their long-term reward by optimizing task offloading and resource allocation. At a certain time t, the reward is determined by the current state of the agent itself and other agents states. For stochastic game formulation $\langle \mathcal{S}, \mathcal{G}, \mathcal{A}, \mathcal{P}, \Delta \rangle$ [21], we can write as follows:

- $\mathcal{S}$ denotes the set of S devices.
- $\mathcal{G}$ denotes the all possible states set.
- $\mathcal{A}_s$ denotes the action of device s. Furthermore, the joint action vector can be given by $\overrightarrow{a} = (a_1, a_2, a_3, \ldots, a_S)^T \in \times_s \mathcal{A}_s$.
- $\mathcal{P}$ denotes the state transition probability.
- Δ_s is the reward function of device s.



DDQN+dueling for joint task offloading and resource allocation

$$\mathbf{P-DR}: \underset{\wp}{\text{minimize}} \ \mathcal{C}(\wp)$$
$$\text{subject to:}$$
$$\wp_{min} \leq \wp_s \leq \wp_{max}, \forall s \in \mathcal{S},$$
$$\sum_{s=1}^{S} \wp_s \leq \wp_{MAX}.$$

$\wp$ → $\wp^*$

CVX optimizer for local computing resource allocation

Fig. 3. Proposed solution approach.

For our system, the end-devices have two kinds of tasks: (a) selecting an edge server for task offloading and (b) resource block selection. Therefore, the action space can be written as:

$$a_{ls}^{r}(t) = \{\kappa_{s,l}(t), \upsilon_{s,r}(t)\}. \tag{21}$$

where $\kappa_{s,l}(t) \in \{0, 1\}$, $\kappa_{s,l}(t) \in \{\kappa_{s,0}(t), \kappa_{s,2}(t), \ldots, \kappa_{s,L-1}(t)\}$ and $\upsilon_{s,r}(t) \in \{0, 1\}$, $\upsilon_{s,r}(t) \in \{\upsilon_{s,0}(t), \upsilon_{s,1}(t), \ldots, \upsilon_{s,R-1}(t)\}$. The total states can be given by the product LR. At a certain time t for a particular device s, the actions of other devices are denoted by $\mathcal{A}_{-s}(t) = \{a_{l1}^{r}(t), \ldots, a_{ls-1}^{r}(t), a_{ls+1}^{r}(t), \ldots, a_{ls}^{r}(t)\}$. It should be noted that the immediate reward $\Delta_s(t) = \Delta(g, \mathcal{A}_s, \mathcal{A}_{-s})$ of every device s is dependent on its action $\mathcal{A}_s$ at the moment and the actions of other devices $\mathcal{A}_{-s}$. Additionally, the stability of the stochastic game can be determined using Nash Equilibrium. A Nash Equilibrium is said to be achieved using the following:

$$\Delta\big(g, a_{ls}^{r*}, a_{-ls}^{r*}\big) \geq \Delta\big(g, a_{ls}^{r}, a_{-ls}^{r*}\big), \ \forall a_{ls}^{r*} \in \mathcal{A} \tag{22}$$

We now model our stochastic game using a finite-state Markov decision process (MDP). The device reward is dependent upon the other device's actions and its current state. As a result, the Markov property is followed by the process. Lets consider $\langle \mathcal{S}, \mathcal{G}, \mathcal{A}, \mathcal{P}, \Delta \rangle$ to determine the MDP. Our solution should be based on maximizing the reward. There are two ways to select a reward for our work. One way is the choose the throughput. Another way could be to take the reciprocal of the cost function. Similar to many works on reinforcement learning, one can use the $\frac{1}{C}$ as a reward. In a typical stochastic environment, we aim to learn a policy $\pi_s^* : \mathcal{G} \rightarrow \mathcal{A}_s$. The aim of this policy $\pi_s^* : \mathcal{G} \rightarrow \mathcal{A}_s$ is to maximize the long-term rewards of agents. In MARL, device share their state space using message passing using back haul communication links or direct wireless links. Maximizing the value-state function $V_i(g, \pi_s, \pi_{-s})$ of every UE in every state yields the optimal policy π_s^*:

$$V_s(g, \pi_s, \pi_{-s}) = \mathbb{E}\left[\sum_{t=0}^{T-1} \gamma^t \Delta_s(g(t), \pi_s(t), \pi_{-s}(t)) \,\middle|\, g(0) = g\right], \tag{23}$$

where the expectation over all potential trajectories produced by the joint policies π_s and π_{-s} is $\mathbb{E}$. Where $\pi_{-s} = (\pi_1, \pi_2, .., \pi_{s-1}, \pi_{s+1}, \ldots, \pi_S)$ indicates the strategies of the agents other than s. The discount factor is given by γ. The value function while considering the Markov property can be given by:

$$V_s(g, \pi_s, \pi_{-s}) = u_s(g, \pi_s, \pi_{-s}) + \gamma \sum_{g' \in \mathcal{G}} \mathcal{P}_{gg'}(\pi_s, \pi_{-s}) V_s\big(g', \pi_s, \pi_{-s}\big), \tag{24}$$

where $\mathcal{P}_{gg'}(\pi_s, \pi_{-s})$ is the state transition probability and $u_s(g_t, \pi_s, \pi_{-s}) = \mathbb{E}[\Delta_s(g, \pi_s, \pi_{-s})]$. For π_s, the following condition should be met if NE exists:

$$V_s\big(g, \pi_s^*, \pi_{-s}^*\big) \geq V_s\big(g, \pi_s, \pi_{-s}^*\big), \quad \forall g \in \mathcal{G}. \tag{25}$$

For mixed-strategy equilibrium based finite games, there exist a Nash equilibrium to satisfy the following Bellman optimality:

$$V_i^*(g,\pi_s,\pi_{-s}) = V_i\big(g,\pi_s^*,\pi_{-s}^*\big) = \max_{a_i'\in\mathcal{A}_i}\Bigg[u_s\big(g,a_s,\pi_{-s}^*\big)+\gamma\sum_{g'\in\mathcal{G}}\mathcal{P}_{gg'}\big(a_s,\pi_{-s}^*\big) V_s\big(g',\pi_s^*,\pi_{-s}^*\big)\Bigg]. \quad (26)$$

Solving the above MDP (i.e., 26) can be performed using Q-learning which is based on finding the optimal Q-value function, i.e., $Q_i^*(g,a_i)$.

$$Q_s^*(g,a_s) = u_s\big(g_t,a_s,\pi_{-s}^*\big) + \gamma\sum_{g'\in\mathcal{G}}\mathcal{P}_{gg'}\big(a_s,\pi_{-s}^*\big)V_i\big(g',\pi_s^*,\pi_{-s}^*\big), \quad (27)$$

where $V_s(g',\pi_s^*,\pi_{-s}^*) = max_{a_s\in\mathcal{A}_s}Q_s^*(g,a_s)$. Rewrite (25) as follows:

$$V_i(g,a_s,\pi_{-s}) = u_s\big(g,a_s,\pi_{-s}^*\big)+\gamma\sum_{g'\in\mathcal{G}}\mathcal{P}_{gg'}\big(a_s,\pi_{-s}^*\big)\max_{a_s'\in\mathcal{A}_s}Q_s^*\big(g',a_s'\big). \quad (28)$$

Obtaining an information about the transition probability (i.e., $\mathcal{P}_{gg'}$) is very challenging. To address this difficulty, the Q-value function $Q_s^*(g,a_s)$ is computed for Q-learning as:

$$Q_s(g,a_s) = Q_s(g,a_s)+\delta\{u_s(g_t,a_s,\pi_{-s}) + \gamma\max_{a_s'\in\mathcal{A}_s}Q_s\big(g',a_s'\big)-Q_s(g,a_s)\}, \quad (29)$$

where the learning rate, represented by δ, aids in calculating the Q value. For fast convergence, the value of δ needs to be chosen carefully. In this work, ϵ greedy strategy is considered. The action with the highest reward is chosen with a probability of $1-\epsilon$, whereas the random action is chosen with a probability of ϵ. Small state and action spaces are better suited for the previously mentioned Q-learning approach. Large action spaces are not a good fit for Q-learning. To overcome this challenge, one can apply Deep Q-network (DQN). DQN is based on two neural networks: (a) the target network and (b) the online network. In the training process, an online network is frequently updated. Such frequent updating of the online network might result in instability. To cater for this, DQN uses a target network, which is updated less frequently to improve the stability. The following loss function is used to update the Q-network.

$$L_s(\theta) = \mathbb{E}_{g,a_s,u_s(g,a_s),g'}\Bigg[\Big(y_s^{DQN}-Q_s(g,a_s;\theta)\Big)^2\Bigg], \quad (30)$$

where $y_s^{DQN} = u_s(g,a_s)+\gamma\max_{a_s'\in\mathcal{A}_s}Q_s(g'a_s';\theta^-)$. θ^- denotes the weights of the target network. For selecting an action a_s, online network $Q_s(g,a_s;\theta)$ is utilized. The weights of the online network are updated frequently. This frequent update of the weights might result in unstable performance. To address this, one can use periodic updates in the target network to bring more stability. Different transitions are stored in memory. In addition to current values, previous values are also used in computation for good results and faster convergence. As DQN uses both online and target networks for performance enhancement, there are still some challenges in terms of over optimistic estimation. To address this, one can use the concept of Double DQN. DDQN is based on a different expression for y_s^{DQN}.

$$y_s^{DDQN} = u_s(g,a_s)+\gamma Q_s\Big(g',\max_{a_s'\in\mathcal{A}_s}Q_s\big(g'a_s';\theta\big);\theta^-\Big). \quad (31)$$

DDQN determines the optimal value $Q_s(g,a_s;\theta)$ using both the target network and the online network. After that, the y_s^{DDQN} is obtained using the discount factor and current reward. Fig. 3 shows the overview of the proposed solution approach. Although DDQN results in better performance, one can use the concept of dueling to further improve the performance of DDQN [21]. For dueling, one can split the last layer of DDQN network into two networks to yield advantage function and value function. These advantage and value functions are used to compute the Q-value. Now, we discuss the complexity of the proposed scheme. Our proposed scheme uses convex optimization and DDQN with dueling. The complexity of convex optimization is reasonable, and it converges within reasonable iterations [22]. On the other hand, in DDQN, mainly the complexity lies in the two neural networks: (a) online network and (b) target network. In our case, we use fully connected neural networks whose complexity is low. Therefore, overall, we can say that our proposed scheme has low complexity.

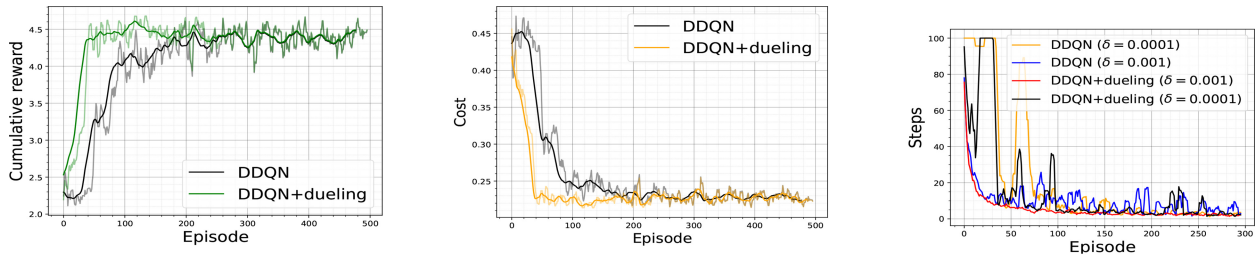
V. Performance Evaluation

A. Simulation Settings

For simulations, we consider different number of consumer electronic devices generated randomly for various runs. Meanwhile, the locations of the edge servers are kept fixed. All the values in our analysis are computed using an average of multiple runs (e.g., 30). The channel bandwidth is taken as $180KHz$. Transmit power ranges between 30dBm and 50dBM for various runs. For path loss, we consider a free space path loss model. The noise power density is considered -174dBm/Hz. Various learning rates (i.e., $\delta = 0.0001, 0.001, 0.00001$) are used for analysis. The number of agents used for various runs and settings are 10, 15, and 20. Our neural network consists of an input layer, two hidden layers, and an output layer. Furthermore, for DDQN+dueling, the last layer is split into two network, i.e., the value function and the advantage function. These two values are then used to compute the Q value. The number of resource blocks considered in a system are equal to the number of agents considered in the system.

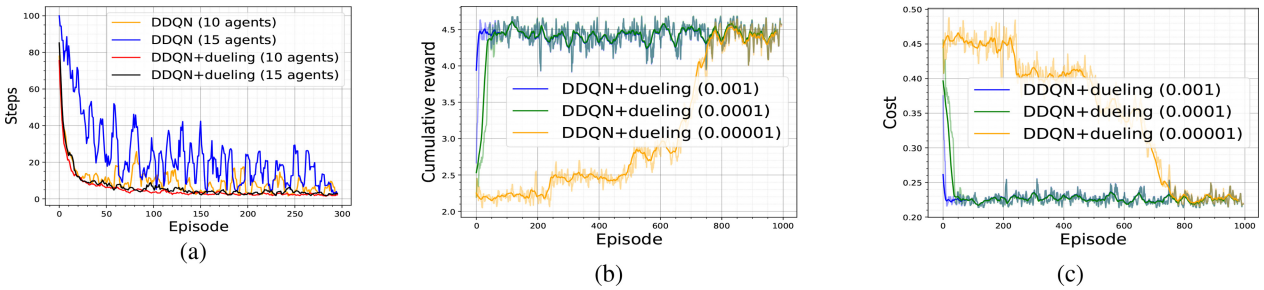
B. Simulation Results

For a fair comparison, we consider various existing baselines as follows:



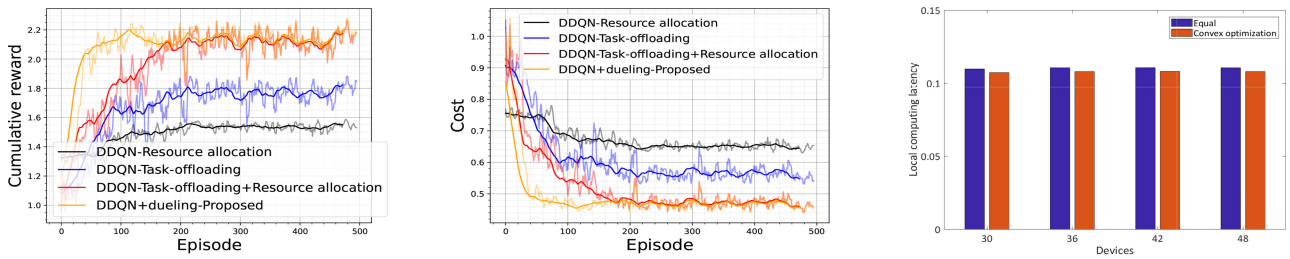
(a) (b) (c)

Fig. 4. (a) Cumulative reward vs. episodes for DDQN and DDQN+dueling, (b) cost vs. episodes for DDQN and DDQN+dueling, and (c) steps needed to reach QoS for various learning rates.



(a) (b) (c)

Fig. 5. (a) steps needed to reach QoS for various number of agents, (b) reward vs. episodes for various learning rates, (c) cost vs. episodes for various learning rates.



(a) (b) (c)

Fig. 6. (a) Reward vs. episodes for DDQN and DDQN+dueling using various existing baselines, (b) cost vs. episodes for DDQN and DDQN+dueling using various existing baselines, (c) cost vs. end-devices for fixed number of edge servers.

- *DDQN+Dueling:* This refers to the proposed scheme that consider dueling along with DDQN for performing joint resource allocation and task offloading.
- *DDQN:* This refers to the proposed scheme that uses DDQN for joint resource allocation and task-offloading.
- *DDQN-Resource Allocation [23]:* In this baseline, resource allocation is performed using DDQN, whereas the task-offloading is considered fixed.
- *DDQN-Task-offloading [24]:* In this baseline, task-offloading is performed using DDQN, whereas the resource allocation is considered fixed.
- *DDQN-Task-offloading+Resource allocation [21]:* In this baseline, both task offloading and resource allocations are performed using DDQN.
- *Convex optimization [22]:* This proposed scheme refers to the use of convex optimization for local computing resource optimization.
- *Equal allocation:* This baseline refers to the use of equal computing resources for end-devices [25].

Fig. 4a shows the cumulative reward vs. episodes for DDQN and DDQN+dueling schemes. For Figs. 4a and 4b, we use 20 iterations within a single episode. In the case of DDQN+dueling, the convergence is fast. Moreover, it is apparent that DDQN+dueling outperforms DDQN in terms of reward. The main reason for this performance enhancement is the stable nature of the DDQN+dueling which divides the last layer by two networks to compute the Q-value, and thus more improved performance. Fig. 4b shows the cost vs. episodes for DDQN and DDQN+dueling. Similar to Fig. 4a, Fig. 4b illustrates that DDQN+dueling outperforms DDQN. The performance in terms of cost converges fast for DDQN+dueling. On the other hand, Fig. 4c and Fig. 5a illustrate the performance of various baselines for attaining a QoS. From Fig. 4c, it is clear that DDQN+dueling attains the QoS in terms of latency very fast compared to DDQN for various learning rates. We use a maximum of 100 steps in every episode. It is evident from both Fig. 4c and Fig. 5a that DDQN needs more steps compared to DDQN+dueling

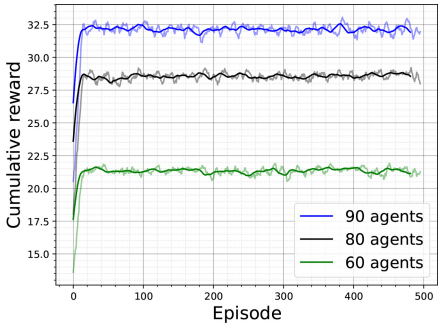


Fig. 7. Reward vs. episodes for various agents for proposed scheme.

to reach QoS by all devices. For learning rate 0.001, both DDQN and DDQN+dueling perform better. Fig. 5a shows that DDQN+dueling outperforms DDQN for different number of agents. For 15 agents, the number of steps is typically higher compared to 10 agents. The reason for this lies in the fact that for more agents, the steps will be generally high. Furthermore, the reason for improved performance of DDQN+dueling compared to DDQN is the consideration of resource allocation, task-offloading and dueling.

Now, we study the effect of variations in learning rate on the performance of DDQN and DDQN+dueling. Figs. 5b shows reward vs. episodes for various learning rates. Note here that the steps used in every episode are 20. We tested for learning rates, 0.001, 0.0001, and 0.00001. For the learning rate 0.00001, the performance is worst. This is because of the fact that neural networks are not learning fast and might stuck in local maxima. Therefore, there is a need for an effective learning rate. For learning rate 0.001, the performance is better and both DDQN and DDQN+dueling achieve faster convergence. If we compare both DDQN and DDQN+dueling, DDQN+dueling achieves faster. Similar to Fig. 5b, Fig. 5c shows the cost vs. episodes for various learning rates using DDQN and DDQN+dueling. This Fig. 5c shows a similar trend of performance to Fig. 5b.

Now, we compare our proposed DDQN+dueling scheme with many existing baselines for a fair comparison. Fig. 6a shows the reward vs. episodes for different baselines. Among all (i.e., DDQN-resource allocation, DDQN-task-offloading, DDQN-task-offloading+resource allocation, DDQN+dueling), DDQN+dueling outperforms and achieves faster convergence. Fig. 6b shows cost vs. episodes for various schemes. Fig. 6b shows the same performance trend as Fig. 6a. Both Figs. 6a and 6b show that the proposed scheme has a better performance compared to all schemes, and thus a promising scheme. Finally, we evaluate the performance of convex optimization and equal computing resource allocation for a various number of devices. We use 30, 36, 42, and 48 number of devices for fixed 6 edge servers. For all devices (i.e., 30, 36, 42, and 48), the proposed convex optimization scheme outperforms the equal allocation scheme. Finally, we study the scalability (i.e., performance of proposed scheme for high number of agents) in Fig. 7. It is evident that proposed scheme has a stable performance for an increase in number of agents. The cumulative reward is high for more number of agents (i.e., 90 agents). The reason for this high reward lies in the fact that more agents will result in high cumulative reward.

VI. Conclusion

In this paper, we have considered SFL and formulated a cost optimization problem. We considered a QoS constraint for learning in terms of latency to ensure SFL with low latency. Our optimization is based on minimizing the cost of SFL. A decomposition-based scheme is used to solve the formulated problem. The end devices computing resource allocation problem is solved using a convex optimizer, whereas the joint task-offloading and resource allocation problems are solved using DDQN+dueling. We proposed the use of DDQN for joint task-offloading and resource allocation. Although DDQN performs well, we added the concept of dueling to further improve the performance so as to achieve faster convergence. From this work, we concluded that our DDQN+dueling-based solution can be extended to many applications/problems.

References

[1] L. U. Khan, M. Guizani, A. Al-Fuqaha, C. S. Hong, D. Niyato, and Z. Han, "A joint communication and learning framework for hierarchical split federated learning," *IEEE Internet Things J.*, vol. 11, no. 1, pp. 268–282, Jan. 2024.

[2] L. U. Khan, A. Elhagry, M. Guizani, and A. El Saddik, "Edge intelligence empowered vehicular metaverse: Key design aspects and future directions," *IEEE Internet Things Mag.*, vol. 7, no. 1, pp. 120–126, Jan. 2024.

[3] E. Rabieinejad, A. Yazdinejad, A. Dehghantanha, and G. Srivastava, "Two-level privacy-preserving framework: Federated learning for attack detection in the consumer Internet of Things," *IEEE Trans. Consum. Electron.*, vol. 70, no. 1, pp. 4258–4265, Feb. 2024.

[4] C. Thapa, P. C. M. Arachchige, S. Camtepe, and L. Sun, "Splitfed: When federated learning meets split learning," in *Proc. AAAI Conf. Artif. Intell.*, 2022, pp. 8485–8493.

[5] Y. Gao et al., "End-to-end evaluation of federated learning and split learning for Internet of Things," in *Proc. Int. Symp. Rel. Distrib. Syst. (SRDS)*, Shanghai, China, 2020, pp. 91–100.

[6] X. Liu, Y. Deng, and T. Mahmoodi, "Wireless distributed learning: A new hybrid split and federated learning approach," *IEEE Trans. Wireless Commun.*, vol. 22, no. 4, pp. 2650–2665, Apr. 2022.

[7] V. Turina, Z. Zhang, F. Esposito, and I. Matta, "Federated or split? A performance and privacy analysis of hybrid split and federated learning architectures," in *Proc. IEEE 14th Int. Conf. Cloud Comput. (CLOUD)*, 2021, pp. 250–260.

[8] S. Otoum, N. Guizani, and H. Mouftah, "On the feasibility of split learning, transfer learning and federated learning for preserving security in its systems," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 7, pp. 7462–7470, Jul. 2023.

[9] J. Shen et al., "RingSFL: An adaptive split federated learning towards taming client heterogeneity," *IEEE Trans. Mobile Comput.*, vol. 23, no. 5, pp. 5462–5478, May 2024.

[10] W. Fan, P. Chen, X. Chun, and Y. Liu, "Madrl-based model partitioning, aggregation control, and resource allocation for cloud-edge-device collaborative split federated learning," *IEEE Trans. Mobile Comput.*, vol. 24, no. 6, pp. 5324–5341, Jun. 2025.

[11] G. Zhu, Y. Deng, X. Chen, H. Zhang, Y. Fang, and T. F. Wong, "ESFL: Efficient split federated learning over resource-constrained heterogeneous wireless devices," *IEEE Internet Things J.*, vol. 11, no. 16, pp. 27153–27166, Aug. 2024.

[12] S. T. Ahmed, V. V. Kumar, and J. Jeong, "Heterogeneous workload based consumer resource recommendation model for smart cities: Ehealth edge–cloud connectivity using federated split learning," *IEEE Trans. Consum. Electron.*, vol. 70, no. 1, pp. 4187–4196, Feb. 2024.

[13] X. Xiong, K. Zheng, L. Lei, and L. Hou, "Resource allocation based on deep reinforcement learning in IoT edge computing," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 6, pp. 1133–1146, Jun. 2020.

[14] Y. He, Y. Wang, C. Qiu, Q. Lin, J. Li, and Z. Ming, "Blockchain-based edge computing resource allocation in IoT: A deep reinforcement learning approach," *IEEE Internet Things J.*, vol. 8, no. 4, pp. 2226–2237, Feb. 2021.
[15] N. Naderializadeh, J. J. Sydir, M. Simsek, and H. Nikopour, "Resource management in wireless networks via multi-agent deep reinforcement learning," *IEEE Trans. Wireless Commun.*, vol. 20, no. 6, pp. 3507–3523, Jun. 2021.
[16] Y. Chen, Z. Liu, Y. Zhang, Y. Wu, X. Chen, and L. Zhao, "Deep reinforcement learning-based dynamic resource management for mobile edge computing in Industrial Internet of Things," *IEEE Trans. Ind. Informat.*, vol. 17, no. 7, pp. 4925–4934, Jul. 2020.
[17] F. Tang, Y. Zhou, and N. Kato, "Deep reinforcement learning for dynamic uplink/downlink resource allocation in high mobility 5G HetNet," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 12, pp. 2773–2782, Dec. 2020.
[18] M. Chen, Z. Yang, W. Saad, C. Yin, H. V. Poor, and S. Cui, "A joint learning and communications framework for federated learning over wireless networks," *IEEE Trans. Wireless Commun.*, vol. 20, no. 1, pp. 269–283, Jan. 2021.
[19] R. S. Klein, H. Luss, and U. G. Rothblum, "Relaxation-based algorithms for minimax optimization problems with resource allocation applications," *Math. Program.*, vol. 64, pp. 337–363, Mar. 1994.
[20] J. Nievergelt, "Exhaustive search, combinatorial optimization and enumeration: Exploring the potential of raw computing power," in *Proc. Int. Conf. Current Trends Theory Pract. Comput. Sci.*, 2000, pp. 18–35.
[21] N. Zhao, Y.-C. Liang, D. Niyato, Y. Pei, M. Wu, and Y. Jiang, "Deep reinforcement learning for user association and resource allocation in heterogeneous cellular networks," *IEEE Trans. Wireless Commun.*, vol. 18, no. 11, pp. 5141–5152, Nov. 2019.
[22] S. P. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge, U.K.: Cambridge Univ. Press, 2004.
[23] A. Iqbal, M.-L. Tham, and Y. C. Chang, "Double deep Q-network-based energy-efficient resource allocation in cloud radio access network," *IEEE Access*, vol. 9, pp. 20440–20449, 2021.
[24] H. Zhai, X. Zhou, H. Zhang, and D. Yuan, "Delay minimization in hybrid edge computing networks: A DDQN-based task offloading approach," *IEEE Trans. Veh. Technol.*, vol. 73, no. 10, pp. 15098–15108, Oct. 2024.
[25] L. U. Khan, M. Guizani, I. Yaqoob, A. Al-Fuqaha, A. Erbad, and Z. Han, "Network virtualization empowered metaverse: A hierarchical matching approach," Techrxiv, Preprints, 2023. [Online]. Available: https://doi.org/10.36227/techrxiv.24049998.v1

Latif U. Khan (Member, IEEE) received the M.S. degree (with Distinction) in electrical engineering from the University of Engineering and Technology, Peshawar, Pakistan, in 2017, and the Ph.D. degree in computer engineering from Kyung Hee University (KHU), South Korea, in 2021. He is author of two books: (a) *Network Slicing for 5G and Beyond* and (b) *Federated Learning for Wireless Networks*. He has reviewed over 200 times for the top ISI-Indexed journals and conferences. He has authored many most popular articles in the leading journals, such as IEEE COMMUNICATIONS SURVEYS AND TUTORIALS and magazines (*IEEE Communication Magazine*, IEEE NETWORK, and *IEEE Wireless Communications Magazine*). His research interests include analytical techniques of optimization and game theory to edge computing, end-to-end network slicing, wireless federated learning, and digital twins. He is the recipient of KHU Best Thesis Award.

Maher Guizani (Member, IEEE) received the master's degree in electrical engineering from Purdue university in 2016. He is currently pursuing the Ph.D. degree in computer engineering with the University of Texas at Arlington. He has experience working in the automotive industry. He worked with General Motors as a Calibration Engineer from 2016 to 2018. He worked with the Embedded Engineering Department, PACCAR from 2018 to 2019. He also worked with the Electrified Semi Truck Department, Meritor from 2020 to 2022.

Sami Muhaidat (Senior Member, IEEE) received the Ph.D. degree in electrical and computer engineering from the University of Waterloo, Waterloo, ON, Canada, in 2006. From 2007 to 2008, he was a Postdoctoral Fellow with the Department of Electrical and Computer Engineering, University of Toronto, ON, Canada. From 2008 to 2012, he was an Assistant Professor with the School of Engineering Science, Simon Fraser University, Burnaby, BC, Canada. He is currently a Professor with Khalifa University, Abu Dhabi, UAE. He was also a Visiting Reader with the Faculty of Engineering, University of Surrey, Guildford, U.K.

Moussa Ayyash (Senior Member, IEEE) received the B.S., M.S., and Ph.D. degrees in electrical and computer engineering. He is currently a Professor with the Department of Computing, Information, and Mathematical Sciences and Technology, Chicago State University, Chicago. He is also the Director of the Center of Information and Security Education and Research.